

mmap()函数在进程的虚拟内存空间中映射出一块内存区域，用以对应指定的一个文件，该内存区域上的数据跟对应的文件的数据是一一对应的，并在一开始的时候用文件的内容来初始化这片内存。

功能	内存映射	
头文件	#include <sys/mman.h>	
原型	void * mmap (void *addr, size_t length, int prot, int flags, int fd, off_t offset);	
参数	addr:	映射内存的起始地址。 如果该参数为 NULL ，则系统将会自动寻找一个合适的起始地址，一般都使用这个值。 如果该参数不为 NULL ，则系统会以此为依据来找到一个合适的起始地址。在 LINUX 中，映射后的内存起始地址必须是页地址的整数倍。
	length:	映射内存大小。
	prot:	映射内存的保护权限。 PROT_EXEC ：可执行。 PROT_READ ：可读。 PROT_WRITE ：可写。 PROT_NONE ：不可访问。
	flags:	当有多个进程同时映射了这块内存时，该参数可以决定在某一个进程内使映射内存的数据发生变更是否影响其他进程，也可以决定是否影响其对应的文件数据。 以下两个选项互斥： MAP_SHARED ：所有的同时映射了这块内存的进程对数据的变更均可见，而且数据的变更会直接同步到对应的文件（有时可能还需要调用 msync() 或者 munmap() 才会真正起作用）。 MAP_PRIVATE ：与 MAP_SHARED 相反，映射了这块内存的进程对数据的变更对别的进程不可见，也不会影响其对应的文件数据。 以下选项可以位或累加： MAP_32BIT ：在早期的 64 位 x86 处理器上，设置这个选项可以将线程的栈空间设置在最低的 2GB 空间附近，以便于上下文切换时得到更好的表现性能，但现代的 64 位 x86 处理器本身已经解决了这个问题，因此这个选项已经被弃用了。 MAP_ANON ：等同于 MAP_ANONYMOUS ，已弃用。 MAP_ANONYMOUS ：匿名映射。该选项使得该映射内存不与任何文件关联，一般来讲参数 fd 和 offset 会被忽略（但是可移植性程序需要将 fd 设置为-1）。另外，这个选项必须跟 MAP_SHARED 一起使用。 MAP_DENYWRITE ：很久以前，这个选项可以使得试图写文件 fd 的进程收到一个 ETXTBUSY 的错误，但是这很快成为所谓“拒绝服务”攻击的来源，因此现在这个选项也被弃用了。 MAP_FIXED ：该选项使得映射内存的起始地址严格等于参数 addr 而

		不仅仅将 addr 当做参考值,这必须要求 addr 是页内存大小的整数倍,由于可移植性的关系,这个选项一般不建议设置。 MAP_GROWSDOWN :使得映射内存向下增长,即返回的是内存的最高地址,一般用于栈。 MAP_HUGETLB :使用“大页”来分配映射内存。关于“大页”请参考内核源代码中的 Documentation/vm/hugetlbpage.txt。 MAP_NONBLOCK :该选项必须与 MAP_POPULATE 一起使用,表示不进行“预读”操作。这使得选项 MAP_POPULATE 变得毫无意义,相信未来的某一天这两个选项会被修改。 MAP_NORESERVE :该选项旨在不为这块映射内存使用“交换分区”,也就是说当物理内存不足时,操作映射内存将会收到 SIGSEGV,而如果允许使用交换分区则可以保证不会因为物理内存不足而出现这个错误。 MAP_POPULATE :将页表映射至内存中,如果用于文件映射,该选项会导致“预读”的操作,因而在遇到页错误的时候也不会被阻塞。 MAP_STACK :在进程或线程的栈中映射内存。 MAP_UNINITIALIZED :不初始化匿名映射内存。
	fd:	要映射的文件的描述符
	offset:	文件映射的开始区域偏移量,该值必须是页内存大小的整数倍,即必须是函数 sysconf(_SC_PAGE_SIZE) 返回值的整数倍。
返回值	成功	映射内存的起始地址
	失败	(void *) -1
备注	无	

图 4-14 函数 `mmap()` 的接口规范